



Network Attacks and Defenses

Cloud Security

CSE4059

Port Scanning

- Port scanning probes transport-layer ports to identify services that appear reachable from the scanner's network path
- It is a reconnaissance technique
 - It gathers information about exposed services, but does not by itself exploit a vulnerability
- An open port may be intentionally exposed
 - Reachability alone does not show whether the application is secure or vulnerable
- Scan results depend on the network path and probe technique

TCP Probe Responses

- After a TCP SYN probe

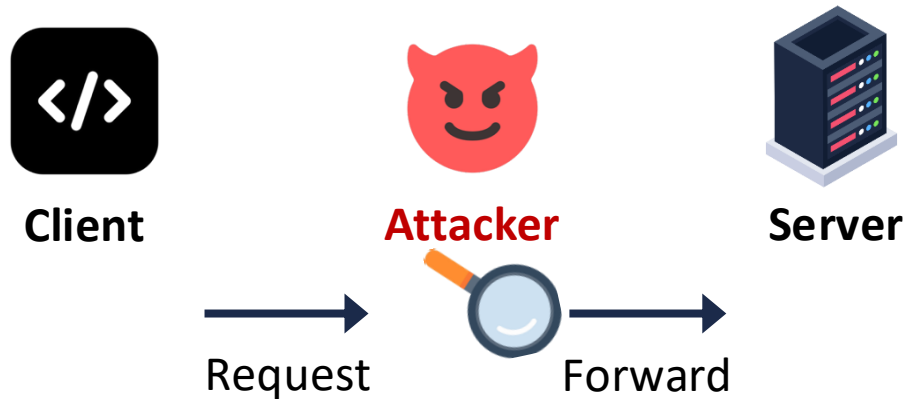
Observed reply	Limited inference
SYN-ACK	Listening endpoint likely
RST	Rejected on this path
No response	Filtering / loss / unavailable

- A **SYN-ACK** is consistent with a listening TCP endpoint
- An **RST** commonly indicates that the connection attempt was rejected
 - It may come from the destination host (or an intervening device)
- **No response** is ambiguous
 - Filtering, packet loss, or an unavailable host can all produce silence
- These observations describe network reachability, not application security

Reducing Service Exposure

- Remove unnecessary services or restrict who can reach them
 - This reduces the number of exposed interfaces available for probing and attack
- Required public services must remain reachable
 - Protect them with secure software, authentication, and authorization
- Moving a service to another port does not remove its vulnerabilities
- Exposure controls restrict reachability, but they do not protect communication on an allowed path.

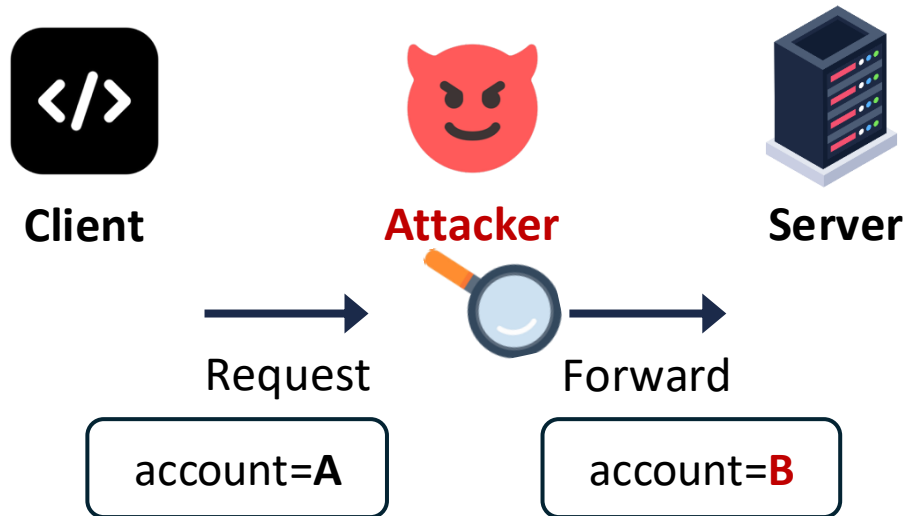
Machine-in-the-Middle Attack



Attacker's goal: inspect or alter communication btw. the client and the service

- A machine-in-the-middle (MITM) attacker
 - Is positioned on the communication path
- Assume the attacker already controls an on-path position
 - Relays traffic between the client and server
- The client still intends to communicate with the legitimate service
- Without cryptographic protection, the attacker may read or modify traffic while forwarding it

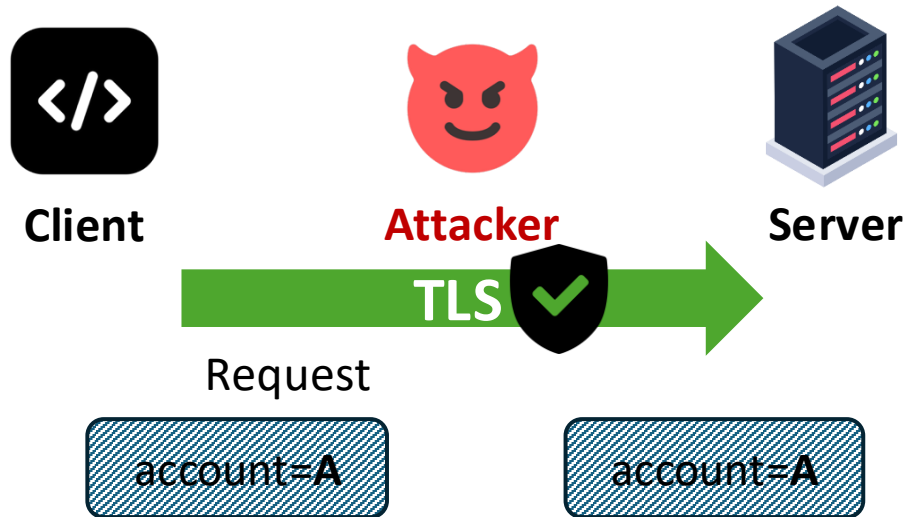
Unprotected Communication



Assumption: No integrity protection

- Client
 - Sends a request containing account=A
 - Attacker
 - Changes account=A to account=B before forwarding the request
 - Without cryptographic integrity protection, the server cannot detect that the message was modified in transit
- The connection needs authenticated cryptographic protection

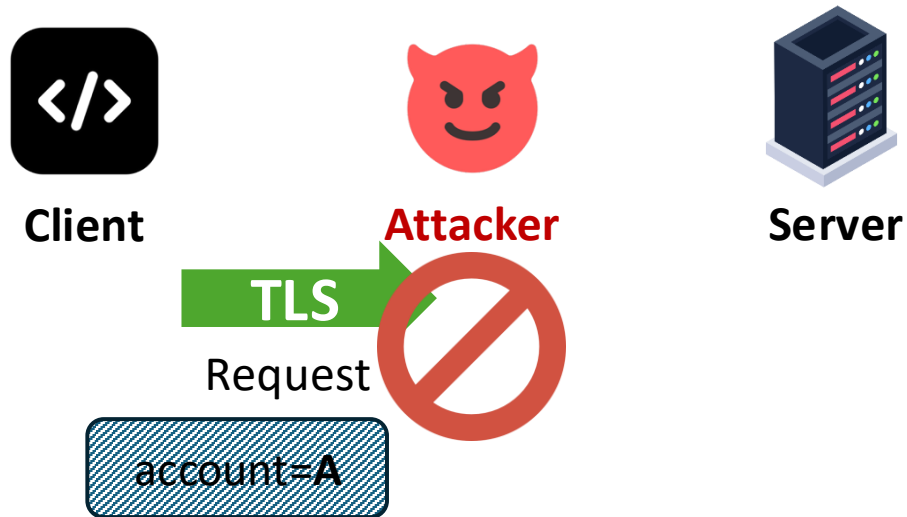
Communication Protected by TLS



Authentication establishes the intended peer
Record protection secures the communication

- Authenticate the intended server during the handshake
 - The client verifies the server's certificate and identity
 - The handshake verifies use of the corresponding private key
 - A MITM attacker cannot simply impersonate the server without credentials accepted for that server
- Protect application data after the handshake
 - TLS endpoints: Client ↔ Authenticated server
 - TLS record protection provides confidentiality and integrity
 - An attacker without the TLS traffic secrets cannot transparently read or modify protected application data

Communication Protected by TLS: Limitation



- Even with correctly authenticated TLS, the attacker is still on the network path
- The attacker cannot read or modify protected application data, but can still **drop or delay** packets
- TLS protects confidentiality, integrity, and peer authentication, but not availability

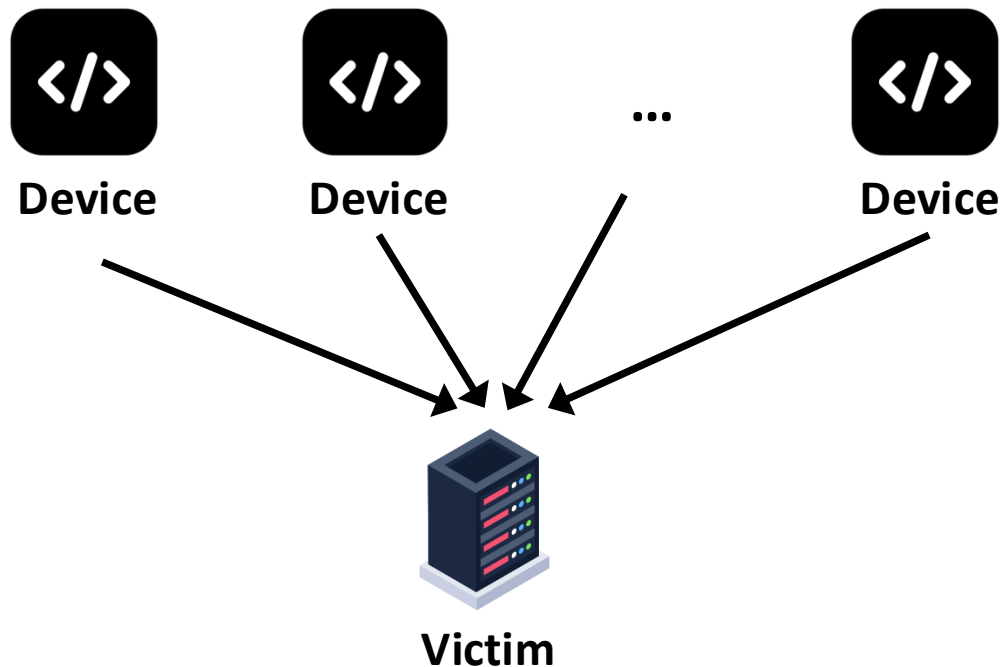
Denial of Service (DoS)

- Denial of Service (DoS) prevents or degrades legitimate access to a service
- A common mechanism is exhausting a resource required to provide the service
- Different DoS attacks can target different resources

Resource	What can run out?	Example
Connection state	Pending-state entries	SYN flooding
Link	Bandwidth	DNS reflection / amplification
Application	Processing / backend capacity	Application request floods

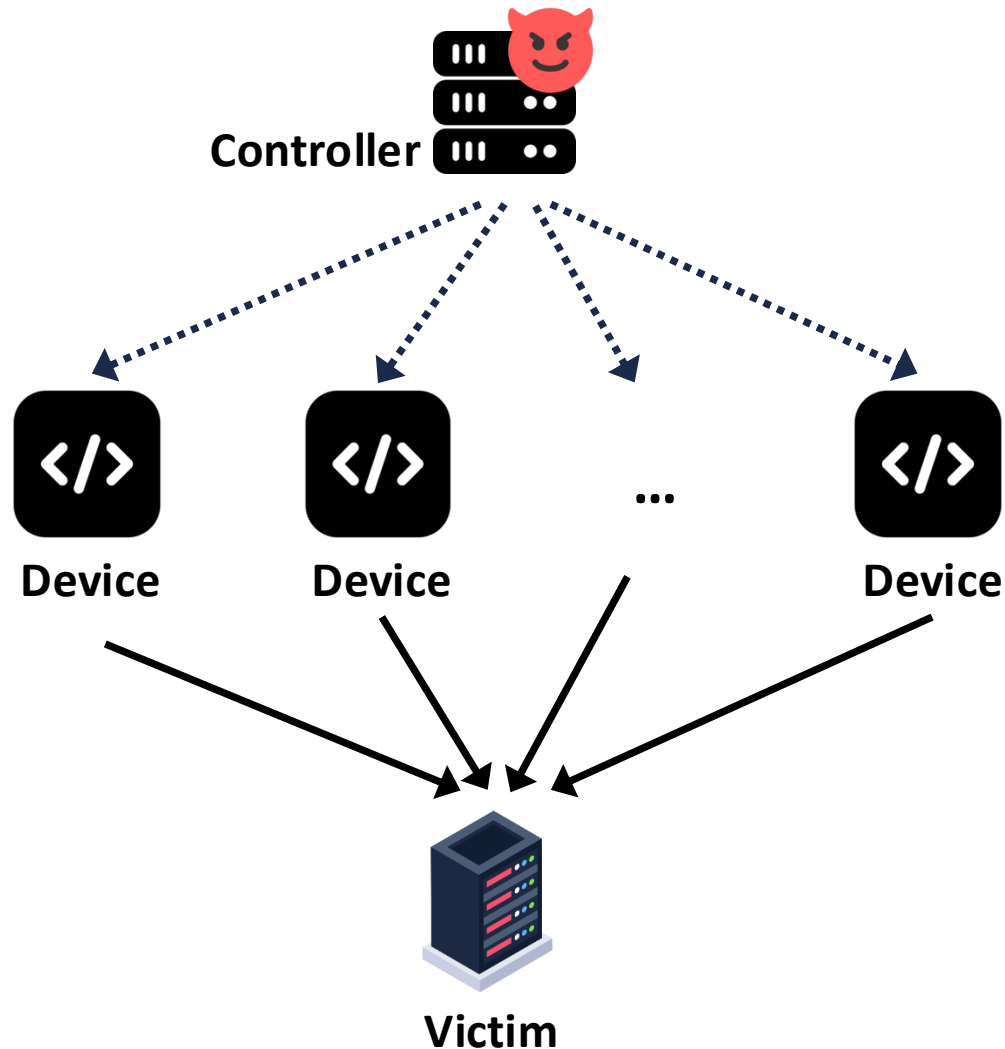
Distributed Denial of Service (DDoS)

Traffic from multiple sources converges



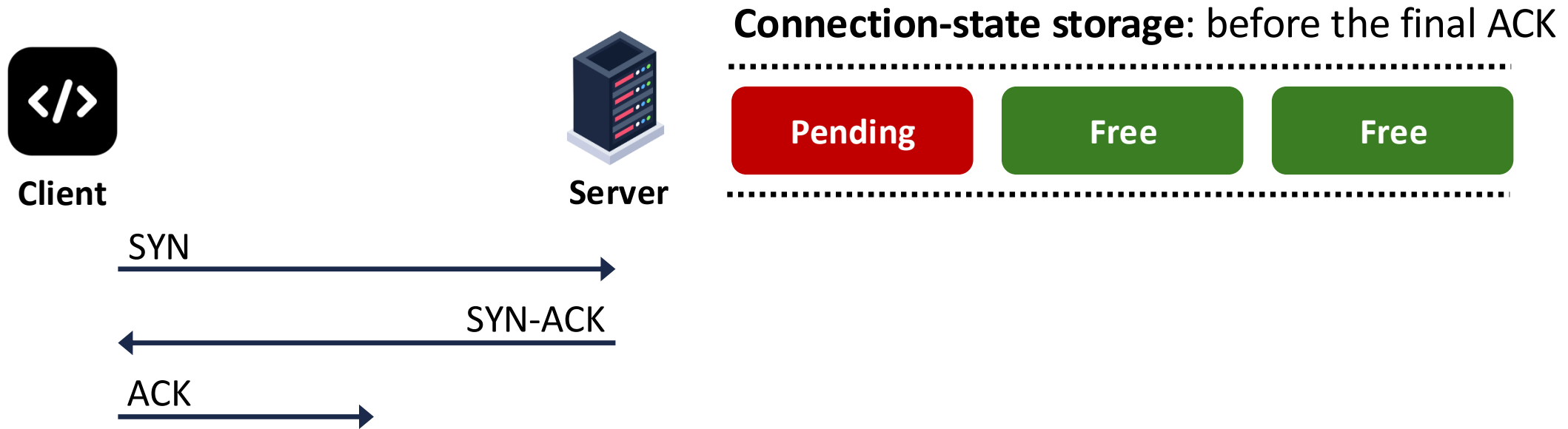
- A DDoS attack uses traffic from many sources to disrupt a service's availability
- The traffic may come from different kinds of systems
 - Compromised devices directly sending attack traffic
 - Reflectors whose responses are redirected toward the victim

DDoS and Botnets



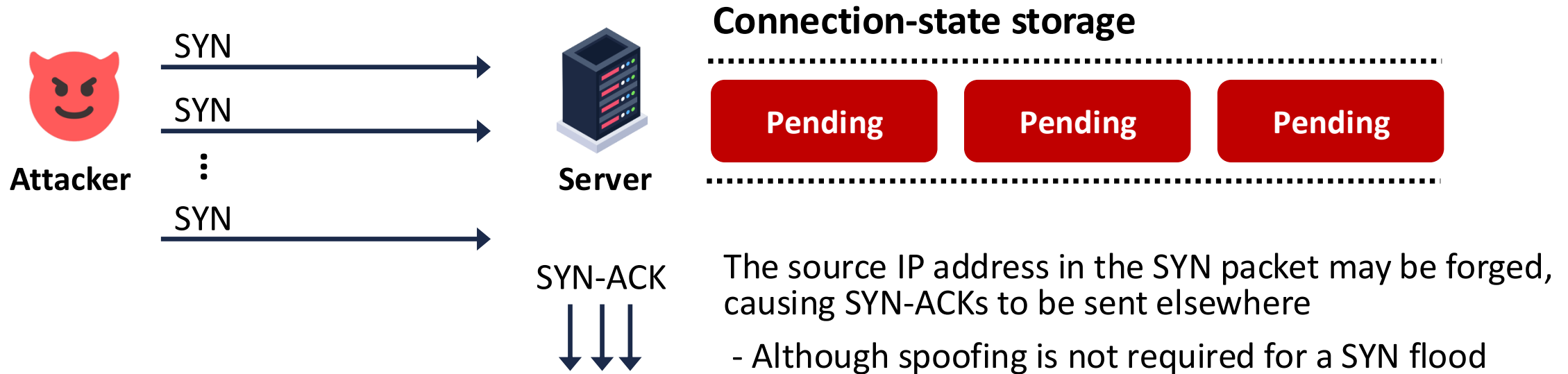
- Botnets are one way to generate attack traffic
- Botnet
 - A group of compromised devices controlled for coordinated activity
- Controller
 - Attacker-controlled infrastructure
 - Sends command-and-control (C2) commands to compromised devices
- The compromised devices then send attack traffic toward the victim
 - C2 traffic and attack traffic are separate flows with different purposes

TCP Pending State



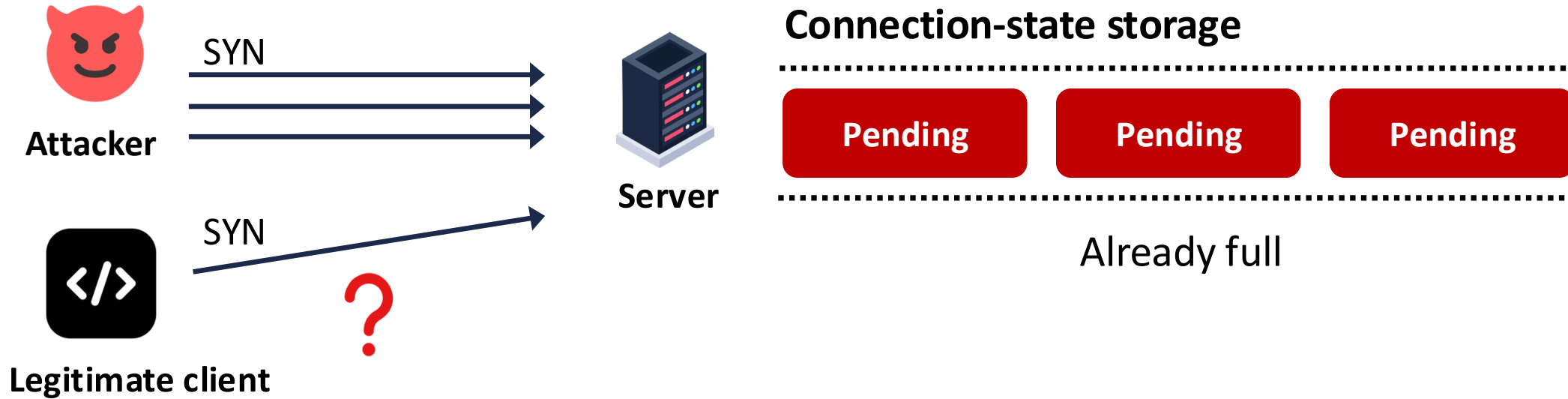
- The server waits for the final ACK before the connection becomes established
- In the conventional model, the server **keeps a pending state** entry while waiting
 - This consumes finite connection-state storage
- The final ACK completes the handshake
 - If the ACK never arrives, the pending entry is eventually removed after a timeout

SYN Flooding



- A SYN flood sends many TCP connection attempts that remain incomplete
 - The server keeps waiting for the final ACKs
- Pending state entries accumulate when new attempts arrive faster than old entries are removed
 - The server's finite connection-state storage can become full

Pending-State Exhaustion



- A legitimate client sends a SYN, but the server's connection-state storage is full
 - Normal connection establishment may be delayed or fail
- Timeouts eventually remove incomplete entries
 - But continuing attack traffic can fill the released space again

SYN Cookies



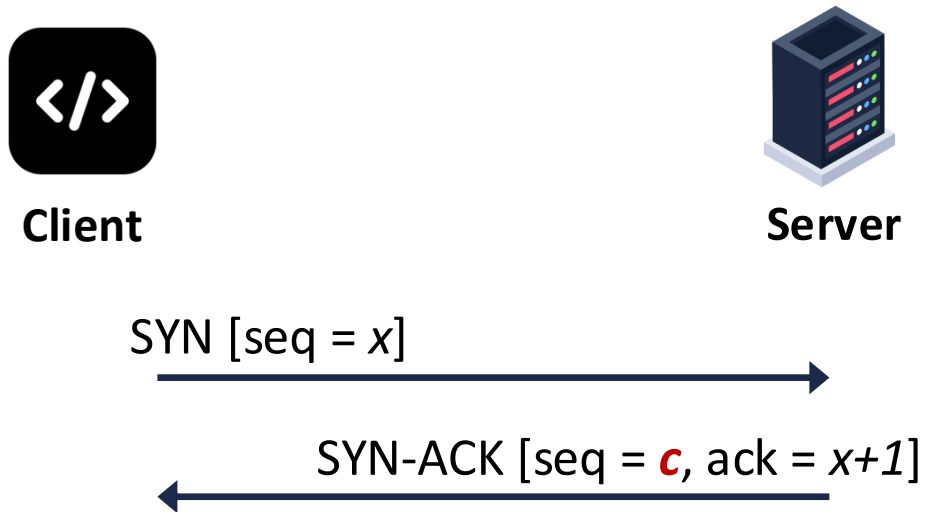
Client



Server

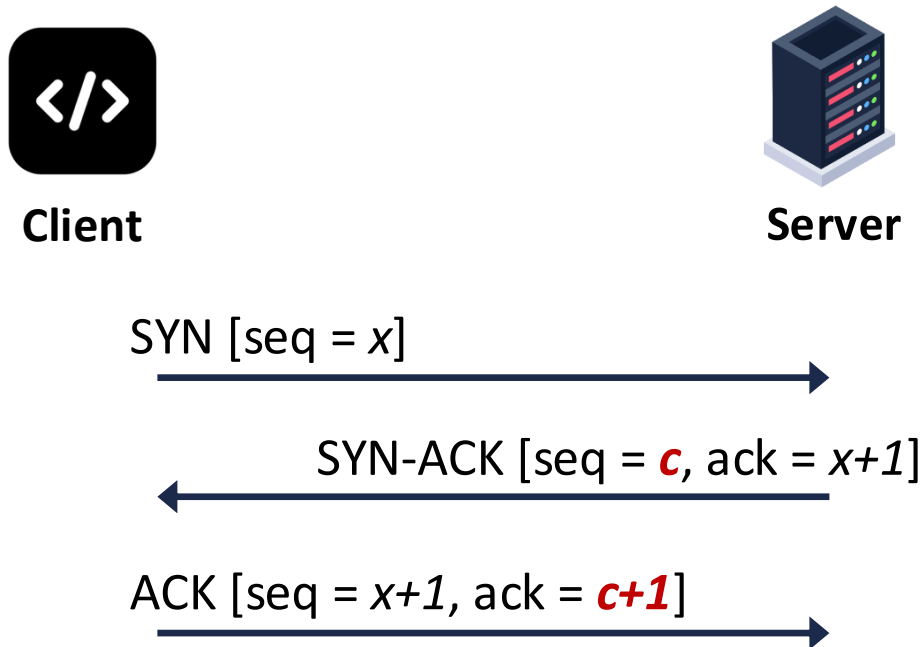
- A SYN cookie is a verifiable value that lets the server delay normal per-connection state allocation until the final ACK returns
 - The server does not keep the usual pending entry for that connection attempt
- The server derives the cookie from
 - Client and server IP addr
 - Client and server TCP ports
 - A recent time value → Prevents old SYN cookies from remaining valid indefinitely
 - A server secret → Prevents attackers from generating valid SYN cookies on their own

SYN Cookies: Delaying State Allocation



- The cookie is used as the server's initial TCP sequence number in the SYN-ACK
- The client sends a SYN with initial sequence number x
- Instead of storing a pending entry, the server generates a SYN cookie **c**
- The server sends
 - SYN-ACK with [seq = **c**, ack = $x+1$]
 - At this point, the server has not allocated the usual per-connection pending state

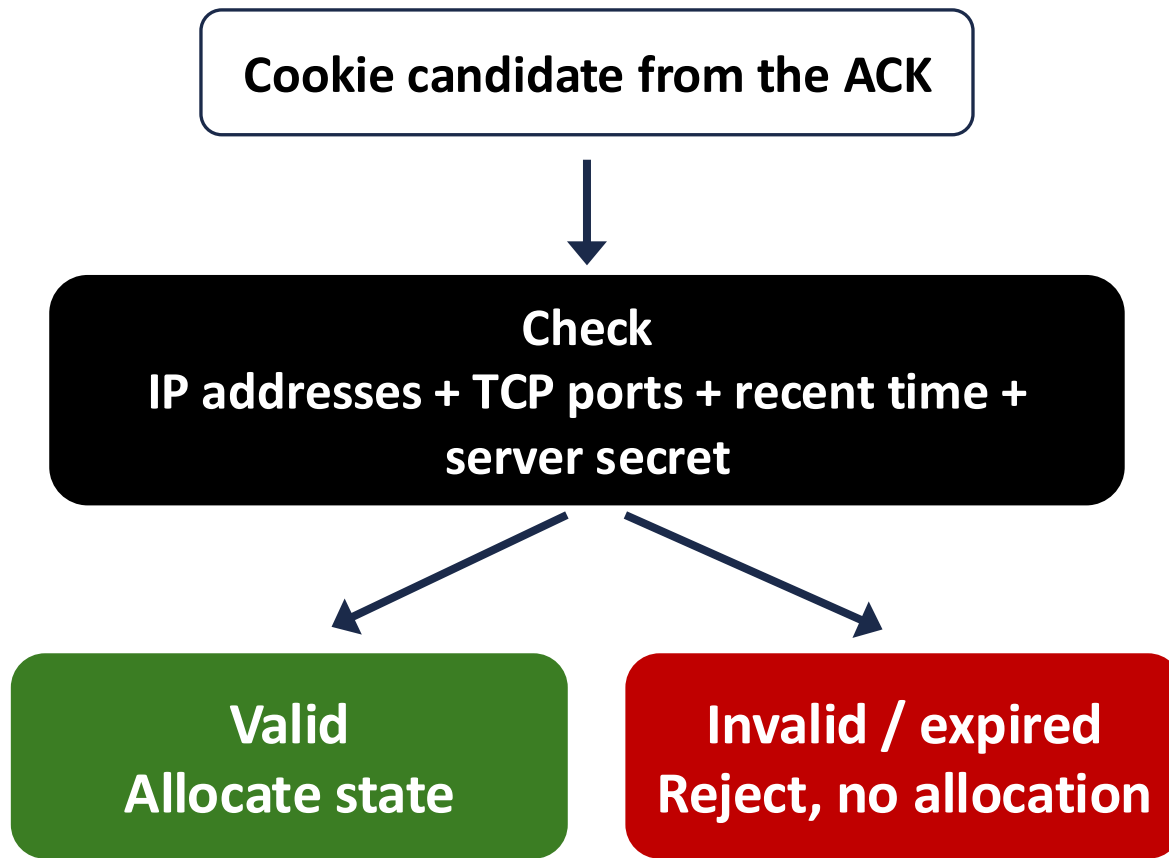
SYN Cookies: Returning the Cookie



- Following normal TCP behavior, the client sends the final ACK
 - ACK with [seq = $x+1$, ack = $c+1$]
- The client does not need to know that c is a SYN cookie.
 - It simply follows the normal TCP acknowledgment rule
- From the returned ACK, the server obtains the cookie candidate
 - $(c + 1) - 1 = c$

Example: no application data during setup

SYN Cookies: Validation and Allocation

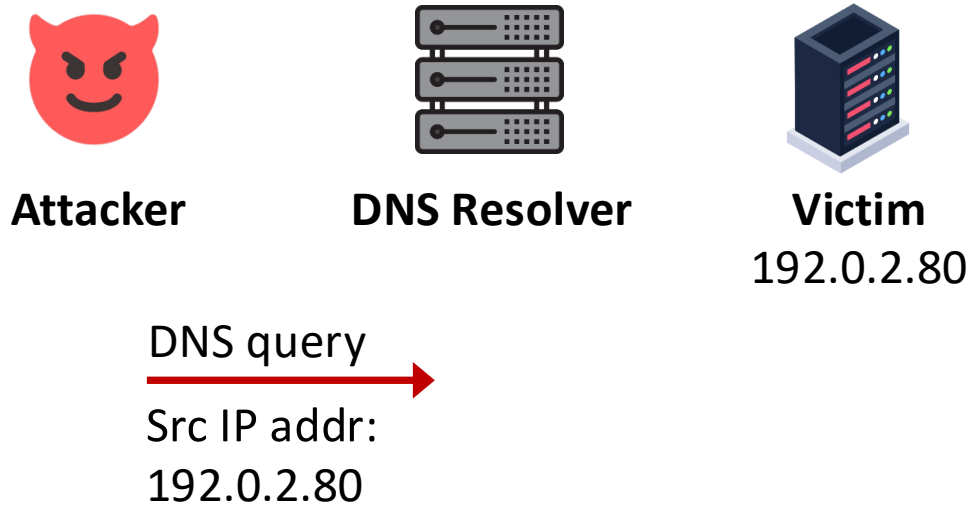


- The server validates the recovered cookie using
 - Client and server IP addresses
 - Client and server TCP ports
 - An acceptable recent time value
 - The server secret
- SYN cookie validation is not user authentication
 - It only verifies that the returned ACK is consistent with a recently generated SYN cookie for that flow

Reflection Attacks

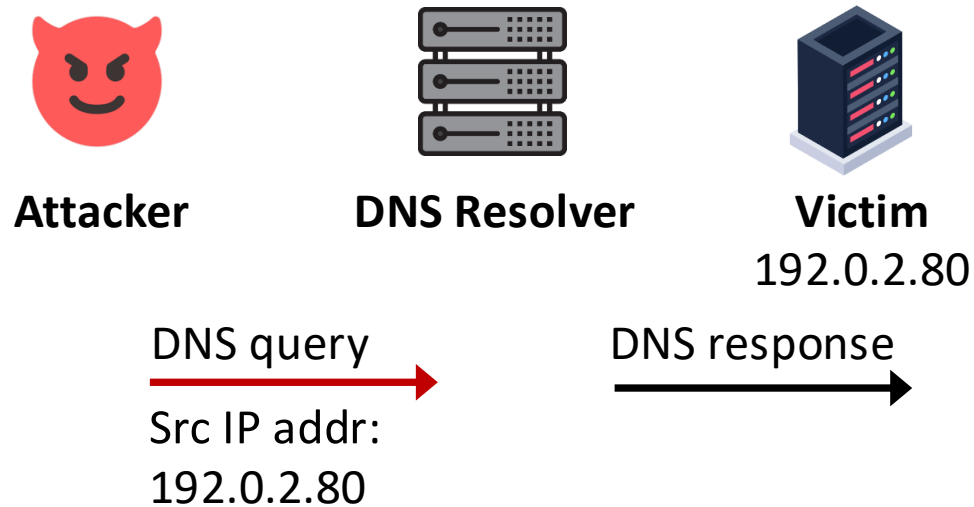
- A reflection attack causes a server to send a response to a victim that **did not send** the original request
- The attacker sends a request with the victim's IP address as the spoofed source address
 - The server sends its response to the victim
- The responding server is called a **reflector**
 - It does not need to be compromised
- Reflection determines where the response goes, not how large it is

DNS Reflection: Request



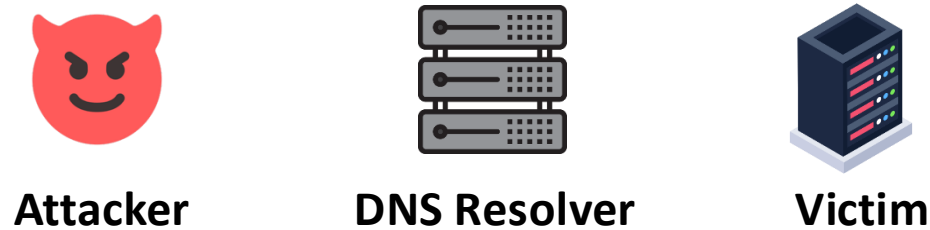
- A DNS resolver is used as a **reflector**
- The attacker sends a DNS query to the DNS resolver
 - Source IP: victim 192.0.2.80
 - Destination: DNS resolver
- The victim did not send the query

DNS Reflection: Response



- The DNS resolver sends the response to the claimed source address
 - The response goes to the victim, not the attacker
- The victim may discard the unsolicited DNS response
 - But the response has already crossed the victim's network link
- The DNS response itself does not need to be false or malicious
 - The problem is the unwanted traffic reaching the victim

DNS Amplification



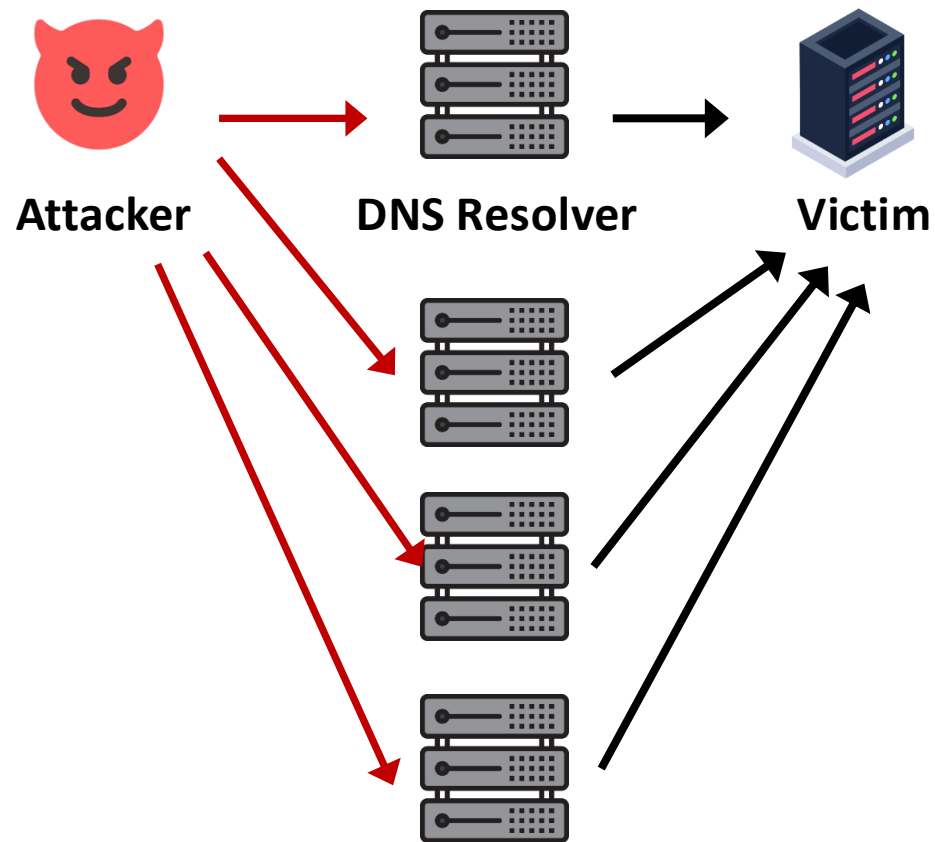
60-byte query → 3,000-byte response

Amplification ratio: $3,000 / 60 = 50x$

*These values are illustrative, not fixed properties of DNS

- Amplification? a small request can trigger a much larger response
- The amplification ratio depends on the query and response
- Reflection and amplification are different concepts
 - Reflection determines where the response goes
 - Amplification determines how much larger the response is than the request

Multiple Reflectors



- The attacker sends spoofed requests to multiple DNS resolvers
 - All claiming the same victim as the source
- Each resolver sends its response to the victim
 - The reflected responses converge on the victim's network
- From the victim's perspective
 - The source addresses belong to the reflectors, not the original attacker
- The reflectors do not need to be compromised

Source-Address Validation

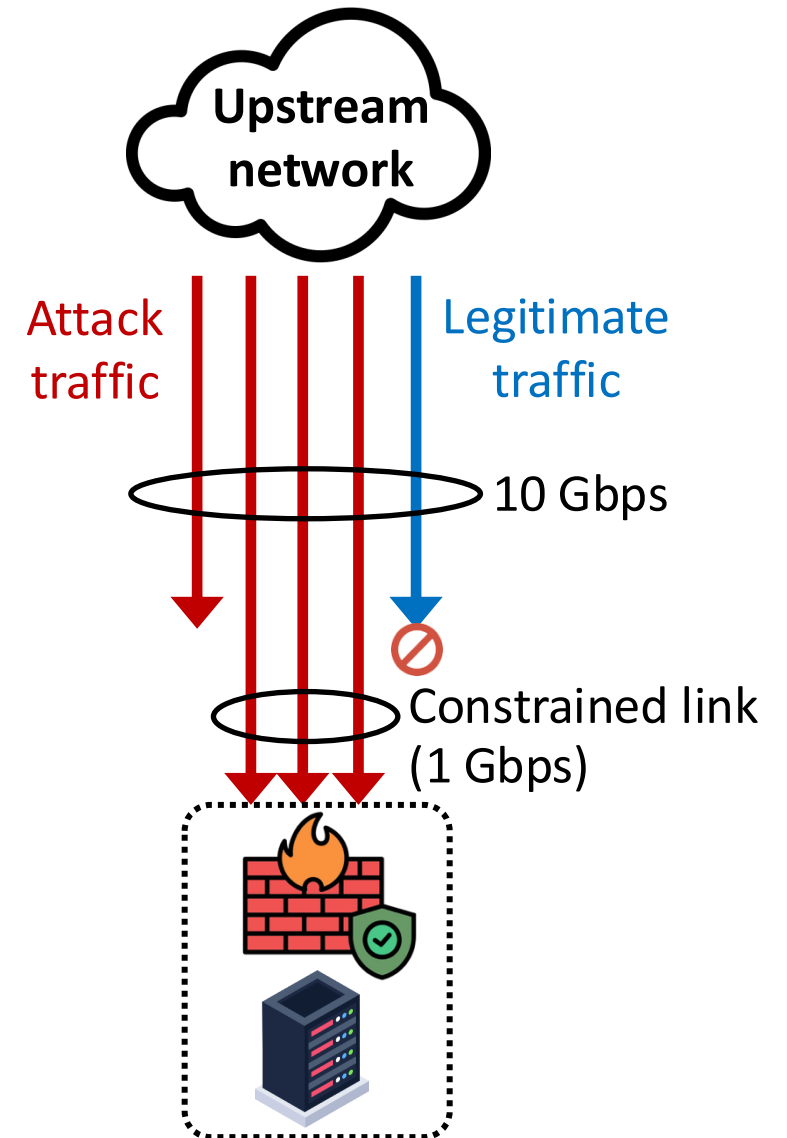
- Source-address validation checks whether a packet's claimed source IP address is valid for the network from which it arrives
- Near the sender, the network can reject packets with an invalid or spoofed source address
 - ex) a packet from the attacker's network should not claim the victim's IP address
- This can stop the spoofed request before it reaches a reflector
 - The reflector therefore does not send a response to the victim
- Limitation?
 - It still does not stop attack traffic sent with legitimate source addresses

Restricting DNS Reflection

- Another defense can act at the reflector (i.e., DNS resolver)
 - rather than near the attacker
- A recursive DNS resolver should provide recursion only to its intended clients
 - An openly accessible recursive resolver can be abused as a reflector
- Restricting resolver access reduces opportunities for DNS reflection attacks
- Appropriate response-rate controls can further reduce abuse
- Authoritative nameservers have a different role
 - They must remain reachable for queries about their published zones

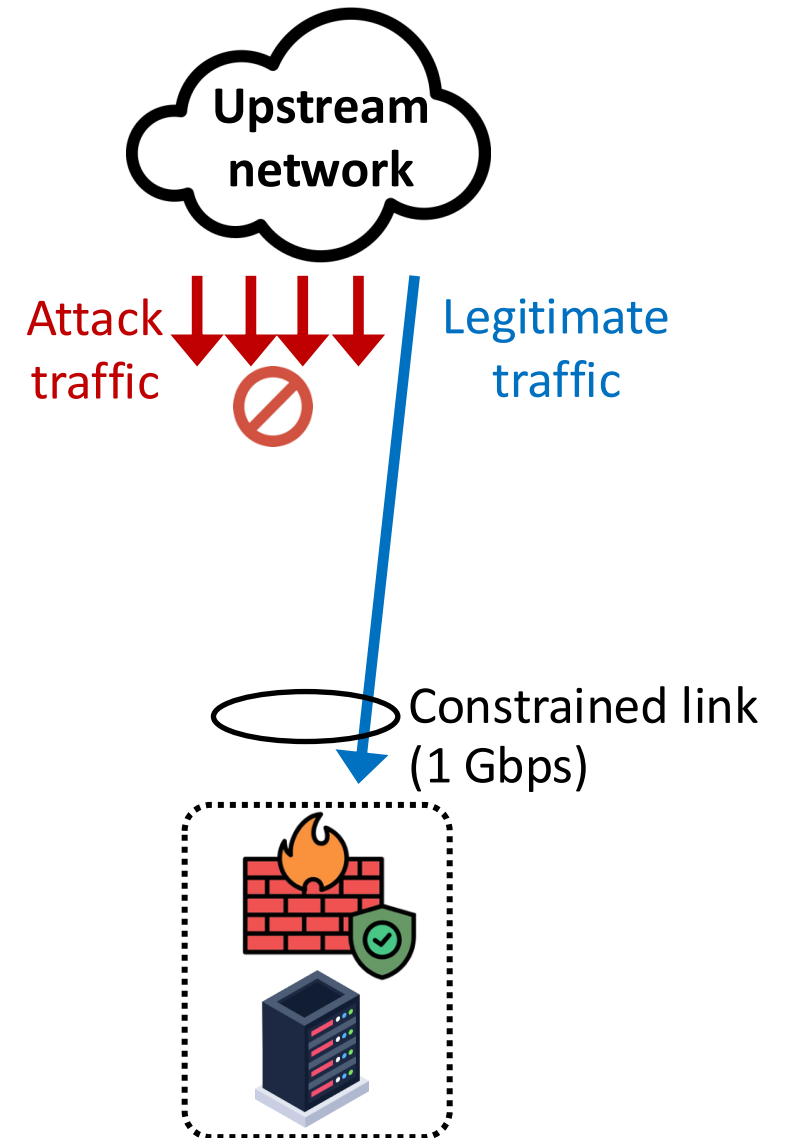
Access-Link Saturation

- Attack traffic and legitimate traffic **share** the same finite-capacity access link
- If incoming traffic exceeds the link capacity, legitimate packets may be delayed or dropped
- A local firewall may correctly drop attack packets
 - But the traffic has already consumed the access-link bandwidth before reaching the firewall
- To protect the link itself, unwanted traffic must be reduced before the bottleneck



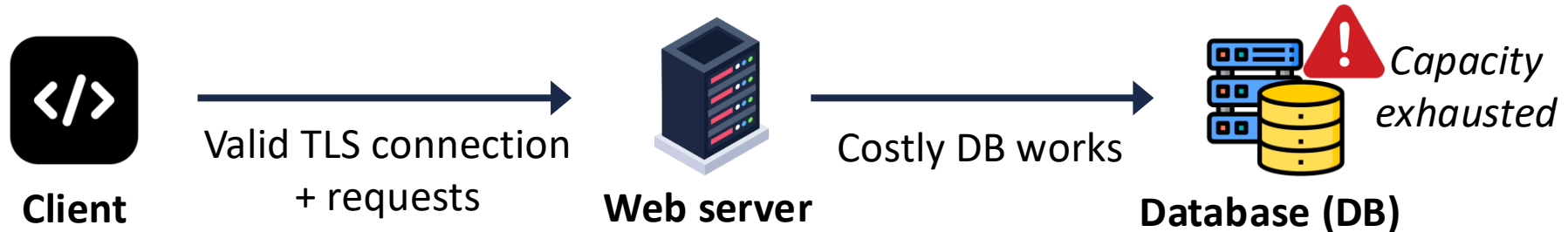
Upstream Mitigation

- Upstream mitigation can reduce unwanted traffic
 - Before it reaches the victim's constrained access link
- An upstream provider or mitigation service can filter or absorb attack traffic at a location with greater capacity
- **Traffic scrubbing** separates suspected attack traffic from traffic judged legitimate
 - Legitimate traffic is forwarded toward the service
- Limitations
 - Sufficient upstream capacity and coordination are required
 - Classification errors may also block legitimate traffic



Application Request Floods

- An application request flood exhausts service resources using requests that may be well-formed and sent over valid connections
- Ex) Each search request triggers database work
 - Accepted requests can exhaust the database capacity even though TCP and TLS work normally



- Not all requests have the same cost
 - An expensive search may require much more processing than a simple request
- The defense must limit application work, not just packet or connection volume
 - ex) limiting expensive operations or concurrent requests

Rate Limits and Tradeoffs

- A rate limit restricts how much activity is accepted over time
 - A concurrency limit restricts how much work can be in progress **at the same time**
- The limit should reflect the cost of the operation
 - A slow or expensive database request may consume much more capacity than a simple request
- Limits must be chosen carefully
 - A limit that is too strict may also reject legitimate demand

Matching Attacks to Controls

- Different attacks affect different resources or security properties
 - The defense should address the actual mechanism and failure point

Problem / Resource	Relevant control
In-transit modification	Authenticated TLS
Pending TCP state exhaustion	SYN cookies
Spoofed reflection	Source-address validation and resolver policy
Saturated access link	Upstream mitigation
Expensive application work	Application limits

- A relevant control does not guarantee complete prevention
 - Each control has a limited scope

Summary

- Port scanning reveals apparent service exposure
- TLS protects communication against on-path reading and modification but does not guarantee availability
- DoS/DDoS attacks can exhaust resources
 - Pending TCP state, access-link capacity, or application processing capacity
- Reflection redirects responses toward a victim, while amplification increases response traffic relative to the triggering request
- An effective defense must match the attack mechanism, affected resource, and defense location